

测试文档

引言

- 1.1 目的
- 1.2 测试概述
- 1.3 参考文献

测试过程

- 单元测试
- 集成测试
- 功能测试

单元测试

- CommentTest
- ProductServiceTest
- StoreTest

集成测试

- 测试简介
- 工具类

- CouponTestUtil
- OrderTestUtil
- ProductTestUtil
- StoreTestUtil
- UserTestUtil

RegisterUseCaseTest

- 测试代码
- 测试截图

StoreUseCaseTest

- 测试代码
- 测试截图

ProductUseCaseTest

- 测试代码

测试截图

OrderUseCaseTest

测试代码

测试截图

CouponUseCaseTest

测试代码

测试截图

功能测试

商店、商品模块

创建商店

注册商店

查看商店列表

创建商品

查看商品列表

增加库存

订单模块

创建订单

支付订单

查看订单

发货与收货

评价

查看评论区

优惠券模块

发布优惠券组

查看优惠券组

使用优惠券支付

其他功能

查询商品

下载订单报表

自主拓展

评论回复

多种优惠券

优惠券可叠加

测试总结

1. 单元测试总结
2. 集成测试总结
3. 功能测试总结

附录

引言

1.1 目的

在软件开发过程中，测试是一个至关重要的环节，其目的不仅仅是找到系统中的缺陷，更在于确保软件系统的质量和可靠性

1.2 测试概述

产品名称	BlueWh ale
版本	v0.0.0
测试标准	RESTful API 测试标 准
测试方法	单元测试, 继 承测试, 单元 测试
硬件环境	华为 matebook16 s通用电子计 算机

软件环境	java, maven, vue, mysql, window11
测试平台	apifox, seecoder

1.3 参考文献

测试过程

单元测试

通过打桩等技术, 完成特定接口针对性的测试

集成测试

串联单元测试内容, 完成对于项目完整功能点的集成测试

功能测试

在前端用不同身份的用户账号, 按照模块和需求进行人工功能测试

单元测试

CommentTest

用于评论测试相关操作的单元测试


```
1  @ExtendWith(SpringExtension.class)
2  @SpringBootTest
3  @AutoConfigureMockMvc
4  public class CommentTest {
5      @Autowired
6      private CommentService commentService;
7
8      @MockBean
9      private CommentRepository commentRepository;
10
11     @MockBean
12     private UserRepository userRepository;
13     @MockBean
14     private SecurityUtil securityUtil;
15
16     private Comment newComment(Integer id, String text) {
17         Comment comment = new Comment();
18         comment.setCreateTime(new Date());
19         comment.setStoreId(0);
20         comment.setOrderId(0);
21         comment.setUserId(0);
22         comment.setRating(0.0);
23         comment.setId(id);
24         comment.setText(text);
25         return comment;
26     }
27
28     @Test
29     public void testCommentOther() throws Exception {
30         CommentVO comment = new CommentVO();
31         comment.setUserId(1);
32         comment.setCommentOnId(2);
33         comment.setOrderId(null);
34         comment.setStoreId(null);
35         comment.setCreateTime(new Date());
36
37         when(commentRepository.findById(2)).thenReturn(java.util.Optional.
of(new Comment()));
38         when(securityUtil.getCurrentUser()).thenReturn(new User());
39
40         Boolean result = commentService.commentOther(comment, 2);
41         Assertions.assertTrue(result);
42     }
43
```

```

44  @Test
45  public void testGetCommentsOn() {
46      List<Comment> comments = new ArrayList<>();
47      comments.add(newComment(1, "Hello"));
48
49      when(userRepository.findById(0)).thenReturn(Optional.of(new User
50  ()));
51      when(commentRepository.findAllByCommentOnId(1)).thenReturn(comment
52  s);
53
54      List<CommentVO> result = commentService.getCommentsOn(1);
55
56      verify(commentRepository, times(1)).findAllByCommentOnId(1);
57      Assertions.assertEquals(1, result.size());
58      Assertions.assertEquals("Hello", result.get(0).getText());
59  }
60
61  @Test
62  public void findComments() {
63      when(commentRepository.findAllByCommentOnId(0)).thenReturn(Arrays.
64  asList(
65      newComment(1, "1"),
66      newComment(3, "3"),
67      newComment(4, "4"),
68      newComment(2, "2")
69  ));
70
71      when(commentRepository.findAllByCommentOnId(1)).thenReturn(Arrays.
72  asList(
73      newComment(5, "5"),
74      newComment(6, "6"),
75      newComment(7, "7"),
76      newComment(8, "8")
77  ));
78
79      when(userRepository.findById(0)).thenReturn(Optional.of(new User
80  ()));
81
82      List<CommentVO> result = commentService.getAllCommentAttached(0);
83      List<CommentVO> result1 = commentService.getAllCommentAttached(1);
84
85      verify(commentRepository, times(1)).findAllByCommentOnId(0);
86      verify(commentRepository, times(2)).findAllByCommentOnId(1);
87
88      Assertions.assertEquals(8, result.size());
89      Assertions.assertEquals(4, result1.size());
90  }

```

ProductServiceTest

用于评论商品相关操作的单元测试

```
1  @SpringBootTest
2  @AutoConfigureMockMvc
3  public class ProductServiceTest {
4      @Autowired
5      private ProductService productService;
6
7      @MockBean
8      private ProductRepository productRepository;
9
10     @MockBean
11     private StoreRepository storeRepository;
12
13     @MockBean
14     private CommentRepository commentRepository;
15
16     @Test
17     public void testCreateProduct() {
18         ProductV0 productV0 = new ProductV0();
19         productV0.setStoreId(1);
20         productV0.setName("Test Product");
21
22         when(productRepository.findByStoreIdAndName(1, "Test Product")).t
henReturn(null);
23         when(productRepository.save(any(Product.class))).thenReturn(new P
roduct());
24
25         Boolean result = productService.create(productV0);
26
27         verify(productRepository, times(1)).findByStoreIdAndName(1, "Tes
t Product");
28         verify(productRepository, times(1)).save(any(Product.class));
29         Assertions.assertTrue(result);
30     }
31
32     @Test
33     public void testCreateProduct_NameAlreadyExists() {
34         ProductV0 productV0 = new ProductV0();
35         productV0.setStoreId(1);
36         productV0.setName("Test Product");
37
38         when(productRepository.findByStoreIdAndName(1, "Test Product")).t
henReturn(new Product());
39
40     }
```

```

41         Assertions.assertThrows(BlueWhaleException.class, () -> productService.create(productV0));
42     }
43
44     @Test
45     public void testAddStock() {
46         Product product = new Product();
47         product.setId(1);
48         product.setStock(10);
49
50         when(productRepository.findById(1)).thenReturn(java.util.Optional.of(product));
51         when(productRepository.save(any(Product.class))).thenReturn(new Product());
52
53         Boolean result = productService.addStock(1, 5);
54
55         verify(productRepository, times(1)).findById(1);
56         verify(productRepository, times(1)).save(any(Product.class));
57         Assertions.assertTrue(result);
58         Assertions.assertEquals(15, product.getStock());
59     }
60
61     @Test
62     public void testAddStock_ProductNotExists() {
63         when(productRepository.findById(1)).thenReturn(Optional.empty());
64
65         Assertions.assertThrows(BlueWhaleException.class, () -> productService.addStock(1, 5));
66     }
67
68     @Test
69     public void testGetAllProducts() {
70         Store store = new Store();
71         store.setId(1);
72         List<Product> products = new ArrayList<>();
73         products.add(new Product());
74
75         when(storeRepository.findById(1)).thenReturn(java.util.Optional.of(store));
76         when(productRepository.findAllByStoreId(1)).thenReturn(products);
77
78         List<ProductV0> result = productService.getAllProducts(1);
79
80         verify(storeRepository, times(1)).findById(1);
81         verify(productRepository, times(1)).findAllByStoreId(1);
82         Assertions.assertEquals(1, result.size());
83     }

```

```

84     }
85
86     @Test
87     public void testGetAllProducts_StoreNotExists() {
88         when(storeRepository.findById(1)).thenReturn(Optional.empty());
89
90         Assertions.assertThrows(BlueWhaleException.class, () -> productService.getAllProducts(1));
91     }
92
93     @Test
94     public void testGetProduct() {
95         Product product = new Product();
96         product.setId(1);
97
98         when(productRepository.findById(1)).thenReturn(java.util.Optional.of(product));
99
100        ProductVO result = productService.getProduct(1);
101
102        verify(productRepository, times(1)).findById(1);
103        Assertions.assertEquals(1, result.getId());
104    }
105
106    @Test
107    public void testGetProduct_ProductNotExists() {
108        when(productRepository.findById(1)).thenReturn(Optional.empty());
109
110        Assertions.assertThrows(BlueWhaleException.class, () -> productService.getProduct(1));
111    }
112
113    @Test
114    public void testGetRating() {
115        Comment comment1 = new Comment();
116        comment1.setRating(4.5);
117        Comment comment2 = new Comment();
118        comment2.setRating(3.5);
119        List<Comment> comments = Arrays.asList(comment1, comment2);
120
121        when(commentRepository.findAllByProductId(1)).thenReturn(comments);
122
123        RatingVO result = productService.getRating(1);
124
125        verify(commentRepository, times(1)).findAllByProductId(1);
126        Assertions.assertEquals(4.0, result.getRating());

```

```

127         Assertions.assertEquals(2, result.getNumRated());
128     }
129
130     @Test
131     public void testGetRating_NoComments() {
132         when(commentRepository.findAllByProductId(1)).thenReturn(Collections.emptyList());
133
134         RatingV0 result = productService.getRating(1);
135
136         verify(commentRepository, times(1)).findAllByProductId(1);
137         Assertions.assertEquals(0.0, result.getRating());
138         Assertions.assertEquals(0, result.getNumRated());
139     }
140
141     @Test
142     public void testGetComments() {
143         List<Comment> comments = new ArrayList<>();
144         comments.add(new Comment());
145
146         when(commentRepository.findAllByProductId(1)).thenReturn(comments);
147
148         List<Comment> result = productService.getComments(1);
149
150         verify(commentRepository, times(1)).findAllByProductId(1);
151         Assertions.assertEquals(1, result.size());

```

StoreTest

用于测试商店服务有关操作

```
1  @ExtendWith(SpringExtension.class)
2  @SpringBootTest
3  @AutoConfigureMockMvc
4  public class StoreTest {
5      @Autowired
6      private StoreService storeService;
7
8      @MockBean
9      private StoreRepository storeRepository;
10
11     @MockBean
12     private CommentRepository commentRepository;
13
14     @MockBean
15     private ProductRepository productRepository;
16
17     @Test
18     public void testCreateStore() {
19         StoreV0 storeV0 = new StoreV0();
20         storeV0.setName("Test Store");
21
22         when(storeRepository.findByName("Test Store")).thenReturn(null);
23         when(storeRepository.save(any(Store.class))).thenReturn(new Store
24     ());
25
26         Boolean result = storeService.create(storeV0);
27
28         verify(storeRepository, times(1)).findByName("Test Store");
29         verify(storeRepository, times(1)).save(any(Store.class));
30         Assertions.assertTrue(result);
31     }
32
33     @Test
34     public void testCreateStore_NameAlreadyExists() {
35         StoreV0 storeV0 = new StoreV0();
36         storeV0.setName("Test Store");
37
38         when(storeRepository.findByName("Test Store")).thenReturn(new Sto
39     re());
40
41         Assertions.assertThrows(BlueWhaleException.class, () -> storeServ
42     ice.create(storeV0));
43     }
```



```

42     @Test
43     public void testGetStore() {
44         Store store = new Store();
45         store.setId(1);
46
47         when(storeRepository.findById(1)).thenReturn(java.util.Optional.of(
48             store));
49
50         StoreVO result = storeService.getStore(1);
51
52         verify(storeRepository, times(1)).findById(1);
53         Assertions.assertEquals(1, result.getId());
54     }
55
56     @Test
57     public void testGetStore_StoreNotExists() {
58         when(storeRepository.findById(1)).thenReturn(Optional.empty());
59
60         Assertions.assertThrows(BlueWhaleException.class, () -> storeService.getStore(1));
61     }
62
63     @Test
64     public void testGetAllStores() {
65         List<Store> stores = new ArrayList<>();
66         stores.add(new Store());
67
68         when(storeRepository.findAll()).thenReturn(stores);
69
70         List<StoreVO> result = storeService.getAllStores();
71
72         verify(storeRepository, times(1)).findAll();
73         Assertions.assertEquals(1, result.size());
74     }
75
76     @Test
77     public void testGetRating() {
78         Comment comment1 = new Comment();
79         comment1.setRating(4.5);
80         Comment comment2 = new Comment();
81         comment2.setRating(3.5);
82         List<Comment> comments = Arrays.asList(comment1, comment2);
83
84         when(commentRepository.findAllByStoreId(1)).thenReturn(comments);
85
86         RatingVO result = storeService.getRating(1);

```

```

87         verify(commentRepository, times(1)).findAllByStoreId(1);
88         Assertions.assertEquals(4.0, result.getRating());
89         Assertions.assertEquals(2, result.getNumRated());
90     }
91
92     @Test
93     public void testGetRating_NoComments() {
94         when(commentRepository.findAllByStoreId(1)).thenReturn(Collections.emptyList());
95
96         RatingVO result = storeService.getRating(1);
97
98         verify(commentRepository, times(1)).findAllByStoreId(1);
99         Assertions.assertEquals(0.0, result.getRating());
100        Assertions.assertEquals(0, result.getNumRated());
101    }
102
103 }

```

集成测试

测试简介

继承测试层主要进行多次接口测试. 通过连续对于http接口序列的调用来测试系统.

工具类

测试代码所公用流程的工具类, 用来复用测试代码

CouponTestUtil

用于优惠券测试相关操作

```

1      public static CouponSetV0 createLocalCouponSetOk(MockMvc mockMvc, Object
    ctMapper objectMapper, String userToken,
2          StoreV0 store, Coupon
    SetRepository couponSetRepository) throws Exception {
3          CouponSetV0 couponSetV0 = new CouponSetV0();
4          couponSetV0.setCouponType(CouponTypeEnum.SPECIAL);
5          couponSetV0.setTotalNum(500);
6          Calendar calendar = Calendar.getInstance();
7          calendar.set(Calendar.YEAR, 2025);
8          calendar.set(Calendar.MONTH, Calendar.JANUARY);
9          calendar.set(Calendar.DAY_OF_MONTH, 1);
10         Date date = calendar.getTime();
11         couponSetV0.setExpireTime(date);
12         mockMvc.perform(MockMvcRequestBuilders.post("/api/coupons")
13             .contentType("application/json")
14             .header("token", userToken)
15             .content(objectMapper.writeValueAsString(couponSet
    V0)))
16             .andExpect(status().isOk())
17             .andExpect(jsonPath("$.code", is("000")));
18         List<CouponSet> expectedCouponSetList = couponSetRepository.findAl
    l();
19         Assertions.assertNotNull(expectedCouponSetList);
20         CouponSetV0 expectedCouponSetV0 = expectedCouponSetList.get(expect
    edCouponSetList.size() - 1).toV0();
21
22         mockMvc.perform(MockMvcRequestBuilders.get("/api/coupons/" + expec
    tedCouponSetV0.getId() + "/get")
23             .contentType("application/json")
24             .header("token", userToken)
25             .content(objectMapper.writeValueAsString(couponSet
    V0)))
26             .andExpect(status().isOk())
27             .andExpect(jsonPath("$.code", is("000")));
28         Assertions.assertEquals(expectedCouponSetV0.getExpireTime(), coupo
    nSetV0.getExpireTime());
29         return expectedCouponSetV0;
30     }
31
32     public static CouponV0 getCoupon(MockMvc mockMvc, ObjectMapper objectMapp
    er, String userToken, int couponSetId)
33     throws Exception {
34         MvcResult result = mockMvc
35

```

```

36         .perform(MockMvcRequestBuilders.get("/api/coupons/" + couponSetId + "/acquire")
37         .contentType("application/json")
38         .header("token", userToken))
39         .andExpect(status().isOk())
40         .andReturn();
41     JsonNode resultNode = objectMapper.readTree(result.getResponse().getContentAsString()).get("result");
42     Coupon coupon = objectMapper.treeToValue(resultNode, Coupon.class);
43     Assertions.assertEquals(coupon.getSetId(), couponSetId);
44     return coupon.toVO();

```

OrderTestUtil

用于订单测试相关操作

```
1  public static Integer createOrder(MockMvc mockMvc, ObjectMapper objectMap
per, String userToken,
2  int productId) throws Exception {
3      OrderVO orderVO = new OrderVO();
4      orderVO.setDeliveryMethod(DeliveryEnum.PICKUP);
5      orderVO.setNum(1);
6      orderVO.setProductId(productId);
7      MvcResult result = mockMvc
8          .perform(MockMvcRequestBuilders.post("/ap
i/orders").contentType("application/json")
9              .header("token", userToke
n)
10             .content(objectMapper.writ
eValueAsString(orderVO)))
11          .andExpect(status().isOk())
12          .andExpect(jsonPath("$.code", is("000"))).
andReturn();
13      JsonNode resultNode = objectMapper.readTree(result.getResp
onse().getContentAsString()).get("result");
14      return Integer.valueOf(objectMapper.treeToValue(resultNod
e, String.class));
15  }
16
17  public static String payOrder(MockMvc mockMvc, ObjectMapper object
Mapper, String userToken,
18  int orderId) throws Exception {
19      MvcResult result = mockMvc
20          .perform(MockMvcRequestBuilders.post("/ap
i/orders/" + orderId + "/pay?isDirectPay=true")
21              .contentType("application/
json")
22              .header("token", userToke
n)
23              .content(objectMapper.writ
eValueAsString(new ArrayList<Integer>())))
24          .andExpect(status().isOk())
25          .andReturn();
26      return result.getResponse().getContentAsString();
27  }
28
29  public static CommentVO commandOrderOk(MockMvc mockMvc, ObjectMapp
er objectMapper, String userToken,
30  int orderId, Double rating, CommentRepository comm
entRepository) throws Exception {
```

```

31         Comment comment = new Comment();
32         comment.setText("test: " + String.valueOf
((int)(Math.random() * 1000000)));
33         comment.setRating(rating);
34         MvcResult result = mockMvc
35             .perform(MockMvcRequestBuilders.post("/ap
i/orders/" + orderId + "/comment")
36                                     .contentType("application/
json")
37                                     .header("token", userToke
n)
38                                     .content(objectMapper.writ
eValueAsString(comment)))
39             .andExpect(status().isOk())
40             .andReturn();
41         JsonNode resultNode = objectMapper.readTree(result.getResp
onse().getContentAsString()).get("result");
42         assertTrue(objectMapper.treeToValue(resultNode, Boolean.cl
ass));
43         Comment commentExpected = commentRepository.findById
(orderId);
44         assertNotNull(commentExpected);
45         assertEquals(comment.getText(), commentExpected.getText
());
46         return commentExpected.toV0();
47     }

```

ProductTestUtil

用于产品测试相关操作

```

1  public static ProductVO createProductOk(MockMvc mockMvc, ObjectMapper obj
   ectMapper, String userToken,
2  int storeId, String name, Double price, ProductRep
   ository productRepository) throws Exception {
3      ProductVO productVO = new ProductVO();
4      productVO.setName(name);
5      ArrayList<String> urls = new ArrayList<String>();
6      urls.add("http://test.com/" + String.valueOf((int) (Math.r
   andom() * 100000)));
7      productVO.setPhotoUrlList(urls);
8      productVO.setPrice(price);
9      productVO.setStoreId(storeId);
10     productVO.setCategory(CategoryEnum.FOOD);
11     // create
12     mockMvc.perform(MockMvcRequestBuilders.post("/api/product
   s").contentType("application/json")
13         .header("token", userToken)
14         .content(objectMapper.writeValueAsString(p
   roductVO)))
15         .andExpect(status().isOk())
16         .andExpect(jsonPath("$.code", is("000")));
17     Product product = productRepository.findByName(name);
18     assertNotNull(product);
19     assertTrue((product.getPhotoUrlList().size() == 1));
20     assertEquals(urls.get(0), product.getPhotoUrlList().get
   (0));
21     return product.toVO();
22 }
23
24 public static ProductVO getProductOk(MockMvc mockMvc, ObjectMappe
   r objectMapper, String userToken,
25 int productId, ProductRepository productRepositor
   y) throws Exception {
26     MvcResult result = mockMvc.perform(MockMvcRequestBuilders.
   get("/api/products/" + productId)
27         .contentType("application/json")
28         .header("token", userToken))
29         .andExpect(status().isOk())
30         .andExpect(jsonPath("$.code", is("000")))
31         .andReturn();
32     JsonNode readNode = objectMapper.readTree(result.getRespon
   se().getContentAsString()).get("result");
33     ProductVO product = objectMapper.readValue(readNode.traver
   se(), ProductVO.class);

```

```

34         assertNotNull(product);
35         Product productExpected = productRepository.findById(product.getId()).get();
36         assertNotNull(productExpected);
37         assertProductEquals(productExpected.toV0(), product);
38         return product;
39     }
40
41     public static void addStock(MockMvc mockMvc, ObjectMapper objectMapper, String userToken,
42         int productId, int num) throws Exception {
43         // get
44         mockMvc.perform(MockMvcRequestBuilders.post("/api/products/" + productId + "/stock?number=" + num)
45             .contentType("application/json")
46             .header("token", userToken)
47             .andExpect(status().isOk())
48             .andExpect(jsonPath("$.code", is("000"))));
49     }
50
51     public static void assertProductEquals(ProductV0 product1, ProductV0 product2) throws Exception {
52         assertEquals(product1.getPhotoUrlList().size(), product2.getPhotoUrlList().size());
53         int photoSize = product1.getPhotoUrlList().size();
54         for (int i = 0; i < photoSize; i++) {
55             assertEquals(product1.getPhotoUrlList().get(i), product2.getPhotoUrlList().get(i));
56         }
57     }

```

StoreTestUtil

用于商店测试相关操作


```

1  public static StoreV0 createStoreOk(MockMvc mockMvc, ObjectMapper objectMapper, String token,
2  StoreRepository storeRepository, String name) throws Exception {
3      // Create store
4      StoreV0 storeV0 = new StoreV0();
5      storeV0.setName(name);
6      storeV0.setLocation("Test Location" + String.valueOf((int) (Math.random() * 100000)));
7      storeV0.setLogoUrl("https://th.bing.com/th/id/OIP.Eib_F-NYXZeCsu355BnugwHaHa?rs=1&pid=ImgDetMain");
8      mockMvc.perform(MockMvcRequestBuilders.post("/api/stores")
9                  .contentType("application/json")
10                 .header("token", token)
11                 .content(objectMapper.writeValueAsString(storeV0))).andExpect(status().isOk())
12                 .andExpect(jsonPath("$.code", is("000")))
13                 .andExpect(jsonPath("$.result", is(true)));
14      Store storeP0 = storeRepository.findByName(storeV0.getName());
15      Assertions.assertEquals(storeP0.getLocation(), storeV0.getLocation());
16      return storeP0.toV0();
17  }
18
19  public static List<ProductV0> getAllProduct(MockMvc mockMvc, ObjectMapper objectMapper, String userToken,
20  int storeId) throws Exception {
21      // get
22      MvcResult result = mockMvc
23          .perform(MockMvcRequestBuilders.get("/api/products?storeId=" + storeId)
24                  .contentType("application/json")
25                  .header("token", userToken))
26          .andExpect(status().isOk())
27          .andExpect(jsonPath("$.code", is("000"))).andReturn();
28
29      JsonNode readNode = objectMapper.readTree(result.getResponse().getContentAsString()).get("result");
30

```

```

31         List<ProductV0> productList = objectMapper.readValue(readNode
32         .traverse(),
33             new TypeReference<List<ProductV0>>() {
34             });
35         return productList;
36     }

37     public static List<ProductV0> searchProduct(MockMvc mockMvc, ObjectMapper objectMapper, String userToken,
38         int storeId, String name, Double minPrice, Double
39         maxPrice) throws Exception {
40         MvcResult result = mockMvc
41             .perform(MockMvcRequestBuilders
42                 .get("/api/stores/" + storeId + "/search?name=" + name + "&minPrice="
43                     + minPrice + "&maxPrice=" + maxPrice + "&category=FOOD")
44                 .contentType("application/
45                     json")
46                 .header("token", userToken)
47                 .andExpect(status().isOk()).andExpect(jsonPath("$.code", is("000"))).andReturn();
48         JsonNode readNode = objectMapper.readTree(result.getResponse().getContentAsString()).get("result");
49         List<ProductV0> productList = objectMapper.readValue(readNode
50         .traverse(),
51             new TypeReference<List<ProductV0>>() {
52             });
53         return productList;

```

UserTestUtil

用于用户测试相关操作

```
1  public static String createManagerOk(MockMvc mockMvc, ObjectMapper objectMapper, TokenUtil tokenUtil,
2      UserRepository userRepository) throws Exception {
3      // Register
4      UserVO userVO = new UserVO();
5      userVO.setPassword("123456");
6      userVO.setAddress("NJU Manager");
7      userVO.setName("Manager");
8      userVO.setPhone("16766666665");
9      userVO.setRole(RoleEnum.MANAGER);
10     mockMvc.perform(post("/api/users/register")
11         .contentType("application/json")
12         .content(objectMapper.writeValueAsString(userVO)))
13         .andExpect(status().isOk())
14         .andExpect(jsonPath("$.code", is("000")));
15
16     // Login
17     String expectedToken = tokenUtil.getToken(userRepository.findByPhone("16766666665"));
18     assertEquals(tokenUtil.getUser(expectedToken).getPhone(), "16766666665");
19     MvcResult result = mockMvc.perform(MockMvcRequestBuilders.post("/api/users/login")
20         .contentType("application/json")
21         .param("phone", "16766666665")
22         .param("password", "123456")).andExpect(status().isOk())
23         .andExpect(jsonPath("$.code", is("000")))
24         .andReturn();
25     // .andExpect(jsonPath("$.result", is(expectedToken)));
26     JsonNode resultNode = objectMapper.readTree(result.getResponse().getContentAsString()).get("result");
27     String token = objectMapper.treeToValue(resultNode, String.class);
28     System.err.println(result);
29     System.err.println(tokenUtil.getUser(token).getPhone());
30     assertEquals(tokenUtil.getUser(token).getPhone(), "16766666665");
31     return expectedToken;
32 }
33
34 public static String createStaffOk(MockMvc mockMvc, ObjectMapper objectMapper, TokenUtil tokenUtil,
35     UserRepository userRepository, int storeId) throws Exception {
36     // Register
37     UserVO userVO = new UserVO();
38     userVO.setPassword("123456");
```

```

39         userV0.setAddress("NJU Staff");
40         userV0.setName("Staff");
41         userV0.setPhone("16766456666");
42         userV0.setRole(RoleEnum.STAFF);
43         userV0.setStoreId(storeId);
44         mockMvc.perform(post("/api/users/register")
45             .contentType("application/json")
46             .content(objectMapper.writeValueAsString(userV0)))
47             .andExpect(status().isOk())
48             .andExpect(jsonPath("$.code", is("000"))));
49
50         // Login
51         String expectedToken = tokenUtil.getToken(userRepository.findByPhone("16766456666"));
52         mockMvc.perform(MockMvcRequestBuilders.post("/api/users/login")
53             .contentType("application/json")
54             .param("phone", "16766456666")
55             .param("password", "123456")).andExpect(status().isOk())
56             .andExpect(jsonPath("$.code", is("000")))
57             .andExpect(jsonPath("$.result", is(expectedToken)));
58         return expectedToken;
59     }
60
61     public static String createCustomerOk(MockMvc mockMvc, ObjectMapper objectMapper, TokenUtil tokenUtil,
62     UserRepository userRepository) throws Exception {
63         // Register
64         UserV0 userV0 = new UserV0();
65         userV0.setPassword("123456");
66         userV0.setAddress("NJU customer");
67         userV0.setName("customer");
68         userV0.setPhone("16766456706");
69         userV0.setRole(RoleEnum.CUSTOMER);
70         mockMvc.perform(post("/api/users/register")
71             .contentType("application/json")
72             .content(objectMapper.writeValueAsString(userV0)))
73             .andExpect(status().isOk())
74             .andExpect(jsonPath("$.code", is("000"))));
75
76         // Login
77         String expectedToken = tokenUtil.getToken(userRepository.findByPhone("16766456706"));
78         mockMvc.perform(MockMvcRequestBuilders.post("/api/users/login")
79             .contentType("application/json")
80             .param("phone", "16766456706")
81             .param("password", "123456")).andExpect(status().isOk())
82             .andExpect(jsonPath("$.code", is("000")))

```

```

83         .andExpect(jsonPath("$.result", is(expectedToken)));
84         return expectedToken;
85     }

```

RegisterUseCaseTest

测试名称	测试内容	预期结果	测试结果
Register	注册接口	注册成功	正确

测试代码

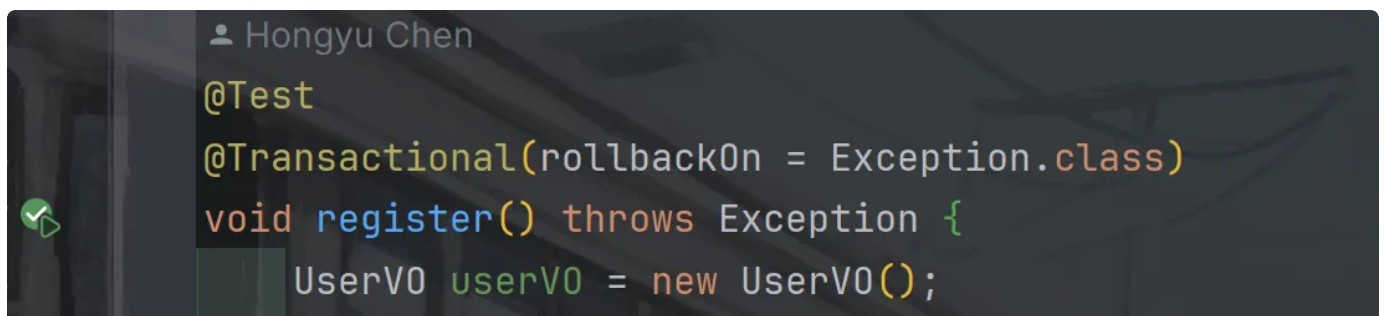
```

RegisterUseCaseTest.java
Java

1    @Test
2    @Transactional(rollbackOn = Exception.class)
3    void register() throws Exception {
4        UserV0 userV0 = new UserV0();
5        userV0.setPassword("123456");
6        userV0.setAddress("NJU");
7        userV0.setName("CHY");
8        userV0.setPhone("16666666666");
9        userV0.setRole(RoleEnum.CUSTOMER);
10       mockMvc.perform(post("/api/users/register", 42L)
11                   .contentType("application/json")
12                   .content(objectMapper.writeValueAsString(userV0)))
13               .andExpect(status().isOk());
14
15       User userP0 = userRepository.findByPhone("16666666666");
16       Assertions.assertThat(userP0.getName()).isEqualTo("CHY");
17   }

```

测试截图



StoreUseCaseTest

测试名称	测试内容	预期结果	测试结果
create	创建商店	商店成功创建	正确
search	搜索商品	搜索结果列表正确	正确

测试代码

```
1      @Test
2      @Transactional(rollbackOn = Exception.class)
3      void create() throws Exception {
4          // Remove existing user
5          userRepository.deleteByPhone("16766666666");
6
7          // Register
8          UserVO userVO = new UserVO();
9          userVO.setPassword("123456");
10         userVO.setAddress("NJU Manager");
11         userVO.setName("Manager");
12         userVO.setPhone("16766666666");
13         userVO.setRole(RoleEnum.MANAGER);
14         mockMvc.perform(post("/api/users/register")
15             .contentType("application/json")
16             .content(objectMapper.writeValueAsString
17                 (userVO)))
18             .andExpect(status().isOk());
19
20         // Login
21         String expectedToken = tokenUtil.getToken(userRepository.
22             findByPhone("16766666666"));
23         mockMvc.perform(MockMvcRequestBuilders.post("/api/users/l
24             ogin")
25             .contentType("application/json")
26             .param("phone", "16766666666")
27             .param("password", "123456")).andExpect(s
28             tatus().isOk())
29             .andExpect(jsonPath("$.result", is(expect
30                 edToken))));
31
32         // Create store
33         StoreVO storeVO = new StoreVO();
34         storeVO.setName("Test Store");
35         storeVO.setLocation("Test Location");
36         storeVO.setLogoUrl("https://th.bing.com/th/id/OIP.Eib_F-N
37             YXZeCsu355BnugwHaHa?rs=1&pid=ImgDetMain");
38         mockMvc.perform(MockMvcRequestBuilders.post("/api/store
39             s")
40             .contentType("application/json")
41             .header("token", expectedToken)
42             .content(objectMapper.writeValueAsString
43                 (storeVO))).andExpect(status().isOk())
44
45         36
```

```

37         .andExpect(jsonPath("$.result", is(true)));
38     e));

39     Store storeP0 = storeRepository.findByName(storeV0.getName());
40     Assertions.assertEquals(storeP0.getLocation(), storeV0.getLocation());
41 }
42
43
44 @Test
45 @Transactional(rollbackOn = Exception.class)
46 void search() throws Exception {
47     String managerToken = UserTestUtil.createManagerOk(mockMvc, objectMapper, tokenUtil, userRepository);
48     StoreV0 storeV0 = StoreTestUtil.createStoreOk(mockMvc, objectMapper, managerToken, storeRepository,
49         "test");
50     String staffToken = UserTestUtil.createStaffOk(mockMvc, objectMapper, tokenUtil, userRepository,
51         storeV0.getId());
52
53     // 范围测试
54     ProductV0 product1 = ProductTestUtil.createProductOk(mockMvc, objectMapper, staffToken, storeV0.getId(),
55         "test10",
56         10.23, productRepository);
57     ProductV0 product2 = ProductTestUtil.createProductOk(mockMvc, objectMapper, staffToken, storeV0.getId(),
58         "test11",
59         20.01, productRepository);
60     ProductV0 product3 = ProductTestUtil.createProductOk(mockMvc, objectMapper, staffToken, storeV0.getId(),
61         "test12",
62         30.78, productRepository);
63     ProductV0 product4 = ProductTestUtil.createProductOk(mockMvc, objectMapper, staffToken, storeV0.getId(),
64         "test13",
65         40.01, productRepository);
66     ProductV0 product5 = ProductTestUtil.createProductOk(mockMvc, objectMapper, staffToken, storeV0.getId(),
67         "test14",
68         50.07, productRepository);
69     List<ProductV0> productList = StoreTestUtil.searchProduct
70         (mockMvc, objectMapper, managerToken,
71         storeV0.getId(),
72         "test1", 20.0, 40.0);
73     assertNotNull(productList);

```



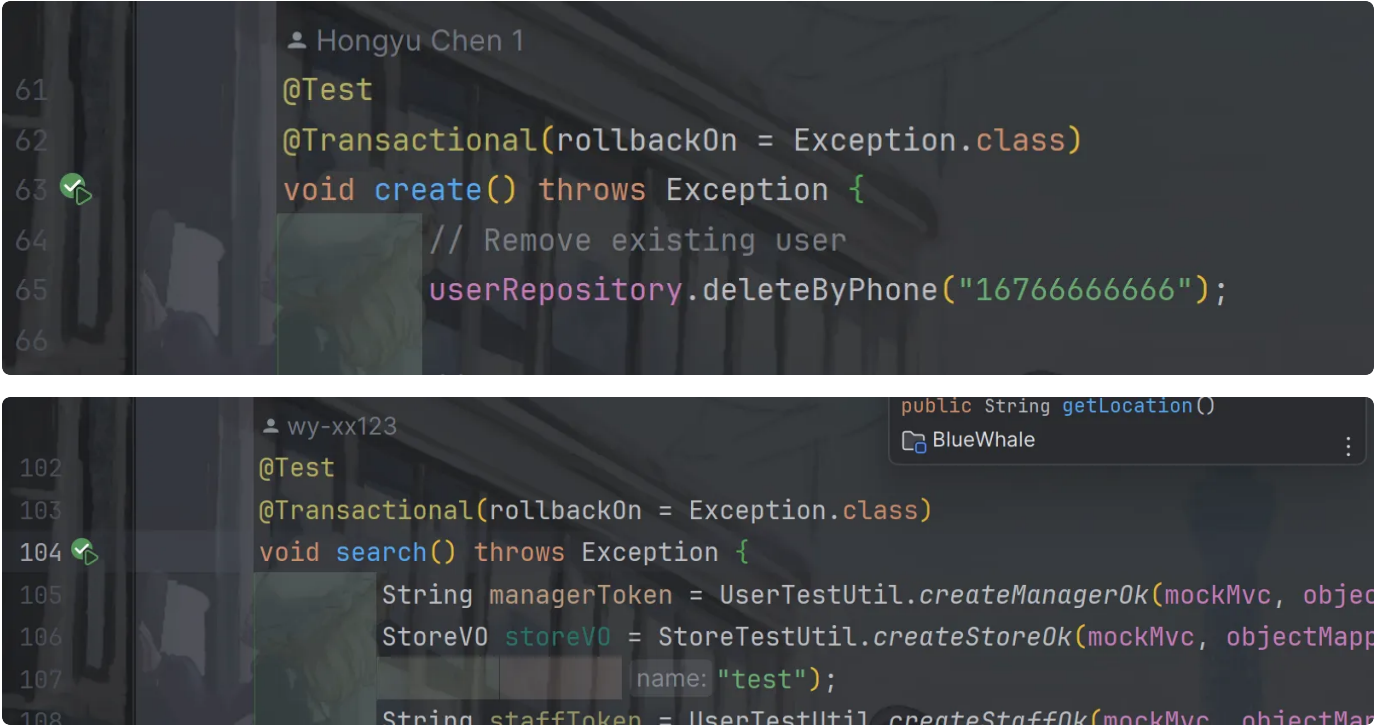
```

73         assertTrue(productList.size() == 2);
74         ProductTestUtil.assertProductEquals(productList.get(0), product2);
75         ProductTestUtil.assertProductEquals(productList.get(1), product3);
76
77         // 模糊搜索测试
78         ProductVO product6 = ProductTestUtil.createProductOk(mockMvc, objectMapper, staffToken, storeV0.getId(),
79             "test-pro",
80             10.00, productRepository);
81         ProductVO product7 = ProductTestUtil.createProductOk(mockMvc, objectMapper, staffToken, storeV0.getId(),
82             "test-proro",
83             10.00, productRepository);
84         ProductVO product8 = ProductTestUtil.createProductOk(mockMvc, objectMapper, staffToken, storeV0.getId(),
85             "ro-test",
86             10.00, productRepository);
87         ProductVO product9 = ProductTestUtil.createProductOk(mockMvc, objectMapper, staffToken, storeV0.getId(),
88             "r.o-test",
89             10.00, productRepository);
90         ProductVO product10 = ProductTestUtil.createProductOk(mockMvc, objectMapper, staffToken,
91             storeV0.getId(),
92             "r.%"'"o-test",
93             10.00, productRepository);
94         ProductVO product11 = ProductTestUtil.createProductOk(mockMvc, objectMapper, staffToken,
95             storeV0.getId(),
96             ".%"'"ro-test",
97             10.00, productRepository);
98         ProductVO product12 = ProductTestUtil.createProductOk(mockMvc, objectMapper, staffToken,
99             storeV0.getId(),
100            "ro",
101            10.00, productRepository);
102         ProductVO product13 = ProductTestUtil.createProductOk(mockMvc, objectMapper, staffToken,
103             storeV0.getId(),
104            "R0",
105            10.00, productRepository);
106         List<ProductVO> productList2 = StoreTestUtil.searchProduct(mockMvc, objectMapper, managerToken,
107             storeV0.getId(),
108            "ro", 5.0, 40.0);

```

```
109         assertTrue(productList2.size() == 5);
110         ProductTestUtil.assertProductEquals(productList2.get(0),
111         product6);
111         ProductTestUtil.assertProductEquals(productList2.get(1),
112         product7);
112         ProductTestUtil.assertProductEquals(productList2.get(2),
113         product8);
113         ProductTestUtil.assertProductEquals(productList2.get(3),
114         product11);
114         ProductTestUtil.assertProductEquals(productList2.get(4),
```

测试截图



ProductUseCaseTest

测试名称	测试内容	预期结果	测试结果
addStock	增加商品库存	商品库存增加成功	正确

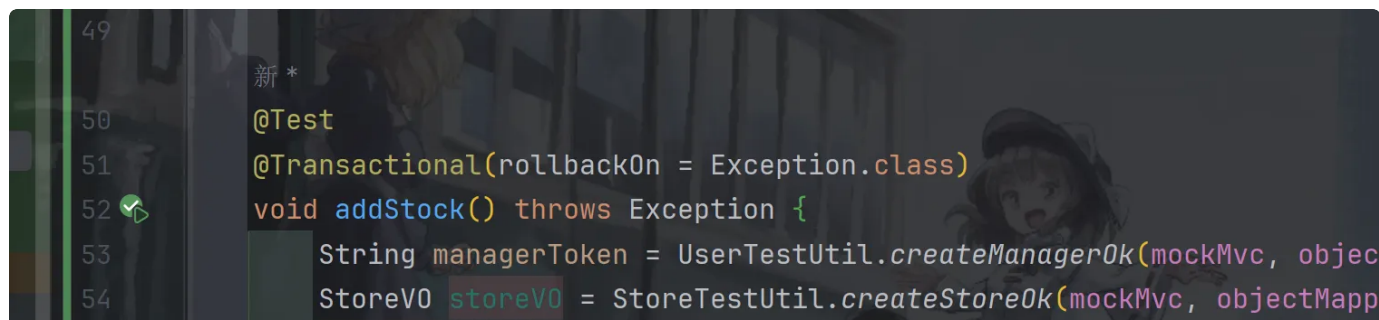
测试代码

```

1  @Test
2      @Transactional(rollbackOn = Exception.class)
3  void addStock() throws Exception {
4      String managerToken = UserTestUtil.createManagerOk(mockMvc, object
Mapper, tokenUtil, userRepository);
5      StoreV0 storeV0 = StoreTestUtil.createStoreOk(mockMvc, objectMappe
r, managerToken, storeRepository,
6          "test");
7      String staffToken = UserTestUtil.createStaffOk(mockMvc, objectMapp
er, tokenUtil, userRepository,
8          storeV0.getId());
9      ProductV0 product1 = ProductTestUtil.createProductOk(mockMvc, obje
ctMapper, staffToken, storeV0.getId(),
10         "test3",
11         10.23, productRepository);
12      ProductV0 product1Get1 = ProductTestUtil.getProductOk(mockMvc, obj
ectMapper, managerToken, product1.getId(), productRepository);
13      // 未添加库存时候 - 库存为0
14      assertEquals(product1Get1.getStock().intValue(), 0);
15      ProductTestUtil.addStock(mockMvc, objectMapper, staffToken, produc
t1Get1.getId(), 3);
16      ProductV0 product1Get2 = ProductTestUtil.getProductOk(mockMvc, obj
ectMapper, managerToken, product1.getId(), productRepository);
17      // 添加了库存时候 - 库存为3
18      assertEquals(product1Get2.getStock().intValue(), 3);
19  }

```

测试截图



OrderUseCaseTest

测试名称	测试内容	预期结果	测试结果
------	------	------	------

pay	支付订单	输出支付宝订单支付界面	成功
-----	------	-------------	----

测试代码

```

▼ OrderUseCaseTest.java
1      @Test
2      @Transactional(rollbackOn = Exception.class)
3      void pay() throws Exception {
4          String managerToken = UserTestUtil.createManagerOk(mockMvc, object
Mapper, tokenUtil, userRepository);
5          StoreV0 storeV0 = StoreTestUtil.createStoreOk(mockMvc, objectMappe
r, managerToken, storeRepository, "test");
6          String staffToken = UserTestUtil.createStaffOk(mockMvc, objectMapp
er, tokenUtil, userRepository,
7              storeV0.getId());
8          ProductTestUtil.createProductOk(mockMvc, objectMapper, staffToke
n, storeV0.getId(), "test", 10.0,
9              productRepository);
10         List<ProductV0> productsList = StoreTestUtil.getAllProduct(mockMvc
, objectMapper, staffToken, storeV0.getId());
11         assertEquals(productsList.size(), 1);
12         ProductV0 productV0 = productsList.get(0);
13         ProductTestUtil.addStock(mockMvc, objectMapper, staffToken, produc
tV0.getId(), 1);
14         String userToken = UserTestUtil.createCustomerOk(mockMvc, objectMa
pper, tokenUtil, userRepository);
15         int orderId = OrderTestUtil.createOrder(mockMvc, objectMapper, use
rToken, productV0.getId());
16         String retHtml = OrderTestUtil.payOrder(mockMvc, objectMapper, use
rToken, orderId);
17         System.out.println("开始进行支付宝支付..."); // here we have html to
ali pay
18         // 支付宝回复html
19         System.out.println(retHtml);
20     }

```

测试截图

```
wy-xx123 *
@Test
@Transactional(rollbackOn = Exception.class)
void pay() throws Exception {
    String managerToken = UserTestUtil.createManagerOk(mockMvc, objectMapper,
    StoreVO storeVO = StoreTestUtil.createStoreOk(mockMvc, objectMapper, m
    String staffToken = UserTestUtil.createStaffOk(mockMvc, objectMapper,
    storeVO.getId());
    ProductTestUtil.createProductOk(mockMvc, objectMapper, staffToken, sto
```

```
开始进行支付宝支付...
<form name="punchout_form" method="post" action="https://openapi-sandbox.dl.alipaydev.com/gateway
<input type="hidden" name="biz_content" value="{&quot;out_trade_no&quot;:&quot;78&quot;;&quot;tot
<input type="submit" value="立即支付" style="display:none" >
</form>
<script>document.forms[0].submit();</script>
2024-06-06 17:03:03.453 INFO 31548 --- [main] o.s.t.c.transaction.TransactionContext
2024-06-06 17:03:03.462 INFO 31548 --- [extShutdownHook] j.LocalContainerEntityManagerFactoryBea
```

CouponUseCaseTest

测试名称	测试内容	预期结果	测试结果
create	创建优惠券组	成功创建优惠券	成功
acquire	在高并发情况下获取优惠券组	并发安全地获取优惠券组	成功

测试代码

```
1      @Test
2      @Transactional(rollbackOn = Exception.class)
3      void create() throws Exception {
4          String managerToken = UserTestUtil.createManagerOk(mockMvc, object
Mapper, tokenUtil, userRepository);
5          StoreV0 storeV0 = StoreTestUtil.createStoreOk(mockMvc, objectMappe
r, managerToken, storeRepository, "test");
6          String staffToken = UserTestUtil.createStaffOk(mockMvc, objectMapp
er, tokenUtil, userRepository, storeV0.getId());
7          CouponSetV0 couponSetV0 = CouponTestUtil.createLocalCouponSetOk(mo
ckMvc, objectMapper, staffToken, storeV0,
8              couponSetRepository);
9      }
10
11      @Test
12      @Transactional(rollbackOn = Exception.class)
13      void acquire() throws Exception {
14          Map<String, String> users = new HashMap<String, String>();
15          for (int i = 0; i < 1000; i++) {
16              String testUid = String.format("%03d", i);
17              String phoneStr = "13727883" + testUid;
18              // register and put tokens into users map
19              users.put(phoneStr, UserTestUtil.createCustomerOk(mockMvc, obj
ectMapper, tokenUtil, userRepository, phoneStr));
20          }
21          // create coupon set
22          String userToken = UserTestUtil.createManagerOk(mockMvc, objectMap
per, tokenUtil, userRepository);
23          StoreV0 storeV0 = StoreTestUtil.createStoreOk(mockMvc, objectMappe
r, userToken, storeRepository, "test");
24          String staffToken = UserTestUtil.createStaffOk(mockMvc, objectMapp
er, tokenUtil, userRepository, storeV0.getId());
25          CouponSetV0 couponSetV0 = CouponTestUtil.createLocalCouponSetOk(mo
ckMvc, objectMapper, staffToken, storeV0,
26              couponSetRepository); // 500 张优惠券
27
28          // get coupon in different thread
29          AtomicBoolean hasException = new AtomicBoolean(false);
30          AtomicInteger successTime = new AtomicInteger(0);
31          ExecutorService executor = Executors.newFixedThreadPool(10);
32          // 创建并提交任务
33          for (int i = 0; i < 10; i++) {
34              final int threadId = i;
35          }
```



```

36     executor.submit(() -> {
37         try {
38             logger.debug("thread " + threadId + "start");
39             for (int j = threadId * 100; j < (threadId + 1) * 10
0; j++) {
40                 String phoneStr = "13727883" + String.format("%03
d", j);
41                 String token = users.get(phoneStr);
42                 Assertions.assertNotNull(token);
43                 CouponVO coupon = CouponTestUtil.getCoupon(mockMv
c, objectMapper, token,
44                     couponSetVO.getId());
45                 successTime.incrementAndGet();
46             }
47         } catch (Exception e) {
48             logger.error(e.getMessage());
49         }
50     });
51 }
52 // 确保所有任务完成之后再继续执行其他代码
53 executor.shutdown();
54 try {
55     // 等待所有任务完成, 最多等待300秒
56     if (!executor.awaitTermination(300, TimeUnit.SECONDS)) {
57         executor.shutdownNow();
58     } // 再次等待所有任务终止
59     if (!executor.awaitTermination(60, TimeUnit.SECONDS)) {
60         System.err.println("线程池未能正确关闭! ");
61     }
62 }
63 } catch (InterruptedException e) {
64     executor.shutdownNow();
65     Thread.currentThread().interrupt();
66 }
67
68 // check number of couponSet
69 CouponSet doneCouponSet = couponSetRepository.findById(couponSetV
0.getId()).orElse(null);
70 Assertions.assertNotNull(doneCouponSet);
71 Assertions.assertEquals(doneCouponSet.getSentNum(),
72     doneCouponSet.getTotalNum());
73 Assertions.assertEquals(doneCouponSet.getSentNum().intValue(),
74     successTime.get());
75 }

```

测试截图

```
9
0
1
2
3
4
wy-xx123
@Test
@Transactional(rollbackOn = Exception.class)
void create() throws Exception {
    String managerToken = UserTestUtil.createManagerOk()
    StoreVO storeVO = StoreTestUtil.createStoreOk(mockI
```

功能测试

商店、商品模块

测试内容	预期结果	测试结果
创建商店	商店创建成功	正确
注册商店	商店注册成功	正确
查看商店列表	商店列表显示成功	正确
创建商品	商品创建成功	正确
查看商品列表	商品列表显示成功	正确
增加库存	库存量增加成功	正确

创建商店



新建商店

商店名

终末番

商店地址

提瓦特

商店Logo



将文件拖到此处或单击此处上传。仅允许上传一份文件。

 background-start.png

创建商店

注册商店

创建一个新账户

昵称

早柚

身份

商家

注册手机号

13555555555

地址

提瓦特

所属商店

终末番

密码

确认密码

手办商铺

终末番

创建账户

去登录

查看商店列表



手办商铺

地址 鸣神大社

参与评分 3 人

★★★★☆



终末番

地址 提瓦特

参与评分 0 人

☆☆☆☆☆

创建商品



新建商品

商品名

品类

单价

商品照片



将文件拖到此处或单击此处上传。

 player-run1.png

创建商品

查看商品列表

蓝鲸在线购物

顾客版

人

目

Q

包

电

←



终末番

地址 提瓦特

商品列表



早柚

品类 娱乐

价格 12 元

库存 0 件

参与评分 0 人

★ ★ ★ ★ ★

增加库存

蓝鲸在线购物

商家版

人

目

Q

包

电

←



早柚

品类 娱乐

价格 12 元

实际库存 0 件

已下单暂未发货 0 件

1000

件

新增库存

评论区 (点击评论展开详情)

暂无评论

39

蓝鲸在线购物商家版

添加库存成功!

用户图标

列表图标

搜索图标

购物车图标

电源图标

←



早柚

品类 娱乐

价格 12 元

实际库存 1000 件

已下单暂未发货 0 件

请输入添加库存数

件

新增库存

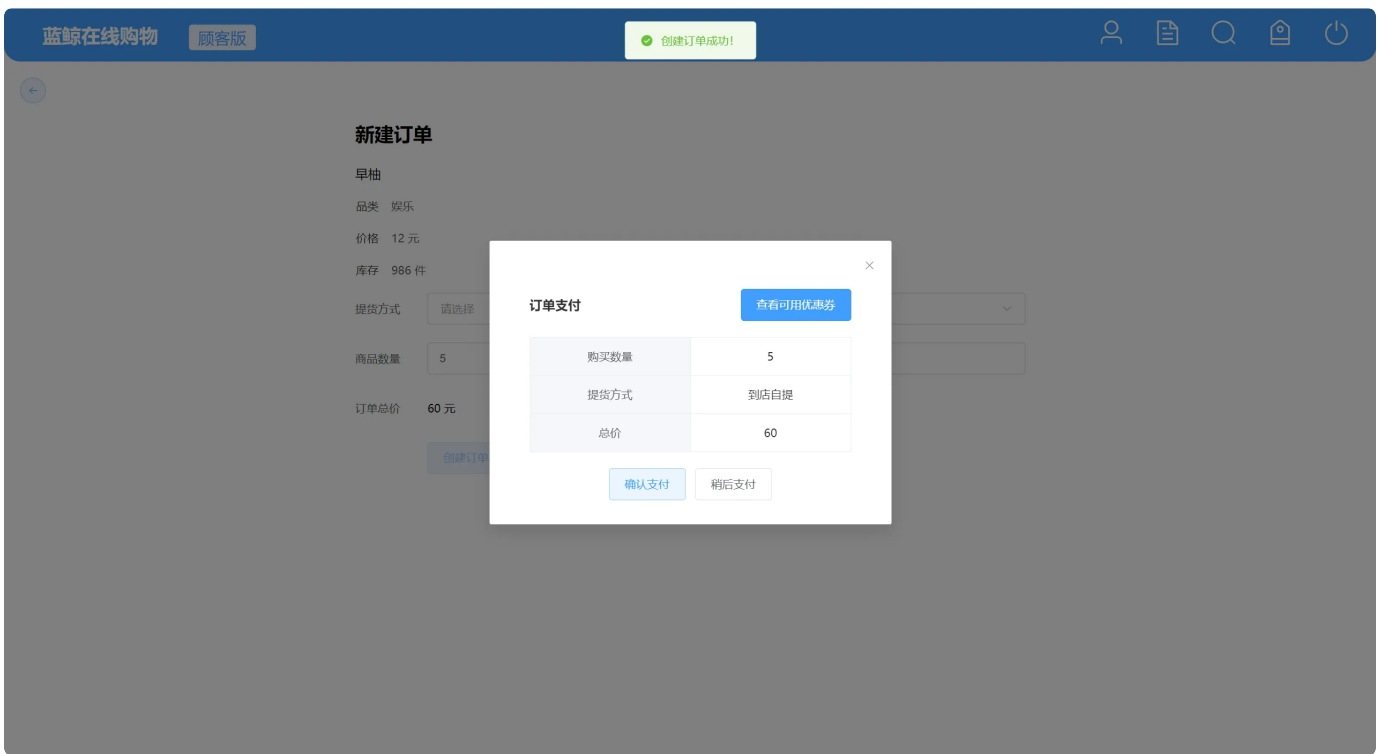
评论区 (点击评论展开详情)

暂无评论

订单模块

测试内容	预期结果	测试结果
创建订单	订单创建成功	正确
支付订单	进入支付宝界面，订单支付成功	正确
查看订单	订单显示成功	正确
发货与收货	订单状态相应改变	正确
评价	订单状态为已完成，且在评论区有评论	正确
查看评论区	评论区显示成功	正确

创建订单



支付订单



正在使用即时到账交易 [?]

bluewhale pay 收款方: qdmfcv9306@sand...

60.00 元

订单详情

扫码支付



使用手机支付宝扫码完成付款
手机支付宝下载 | 如何使用?

登录支付宝账户付款

新用户注册

账户名: 忘记账户名?

dajapn0274@sandbox.com X

支付密码: 忘记密码?

请输入账户的 支付密码，不是登录密码。

下一步

ICP证: 合字B2-20190046



我的收银台



您已成功付款60.00元!

对方将立即收到您的付款。
如果您有未付款信息，查看并继续付款

您可能需要: [查看余额](#) | [消费记录](#) | [我要充值](#)

ICP证: 沪B2-20150087



查看订单

订单号 12 待商家发货

商品 早柚

数量 5 件

总价 60 元

取货类型 到店自提

创建时间 2024-06-07 17:52:03

查看

发货与收货

订单号 6 待商家发货 发货

商品 早柚

数量 1 件

总价 12 元

取货类型 到店自提

创建时间 2024-06-07 11:14:11

查看

订单号 7 待顾客支付

商品 早柚

数量 7 件

总价 84 元

取货类型 到店自提

创建时间 2024-06-07 11:15:18

查看

订单号 12 待商家发货 发货

商品 早柚

数量 5 件

总价 60 元

取货类型 到店自提

创建时间 2024-06-07 17:52:03

查看

订单号 6待商家发货发货 商品 早柚 数量 1 件 总价 12 元 取货类型到店自提 创建时间 2024-06-07 11:14:11 查看	订单号 7待顾客支付 商品 早柚 数量 7 件 总价 84 元 取货类型到店自提 创建时间 2024-06-07 11:15:18 查看	订单号 12待顾客收货 商品 早柚 数量 5 件 总价 60 元 取货类型到店自提 创建时间 2024-06-07 17:52:03 查看
---	---	--

订单号 12待顾客收货收货 商品 早柚 数量 5 件 总价 60 元 取货类型到店自提 创建时间 2024-06-07 17:52:03 查看
--

订单号 12 待顾客评价 评价

商品

旱柚

数量

5 件

总价

60 元

取货类型

到店自提

创建时间

2024-06-07 17:52:03

查看

评价

蓝鲸在线购物

顾客版



订单号 12 待顾客评价 评价

商品

旱柚

数量

5 件

总价

60 元

取货类型

到店自提

创建时间

2024-06-07 17:52:03

查看

评论

×

我很喜欢!

5 / 120

请您为该商品评分

★ ★ ★ ★ ★

目前已有 0 人参与评分

退出 确定

订单号 12 已完成

商品 早柚

数量 5 件

总价 60 元

取货类型 [到店自提](#)

创建时间 2024-06-07 17:52:03

[查看](#)

查看评论区

蓝鲸在线购物

顾客版





早柚

品类 娱乐

价格 12 元

库存 981 件

[购买](#)

评论区 (点击评论展开详情)

我只是个游客

2024-06-07 18:01:22



我很喜欢!

优惠券模块

测试内容	预期结果	测试结果
------	------	------

发布优惠券组	优惠券组发布成功	正确
查看优惠券组	优惠券组显示成功	正确
使用优惠券支付	领取优惠券，使用优惠券支付成功	正确

发布优惠券组

蓝鲸在线购物

CEO版

👤

📄

🔍

🏠

🔌

←

新建优惠券组

优惠券类型

蓝鲸券

优惠券数量

5

过期时间

📅 2024-06-30

创建优惠券组

蓝鲸在线购物

CEO版

创建优惠券组成功!

用户头像

菜单

搜索

购物车

电源

创建优惠券组

可用商店：手办商铺

类型 满减券

优惠 满 100 减 20

发布时间 2024-06-06 20:23:45

过期时间 2024-06-17 00:00:00

总数量 10 张

剩余 7 张

可用商店：手办商铺

类型 蓝鲸券

发布时间 2024-06-06 20:24:06

过期时间 2024-06-18 00:00:00

总数量 10 张

剩余 7 张

所有商店均可用

类型 代金券

发布时间 2024-06-06 20:24:30

过期时间 2024-06-13 00:00:00

总数量 2 张

剩余 0 张

可用商店：手办商铺

类型 代金券

发布时间 2024-06-06 20:37:38

过期时间 2024-06-05 00:00:00

总数量 3 张

剩余 3 张

可用商店：手办商铺

类型 普通打折券

发布时间 2024-06-06 22:09:49

过期时间 2024-06-18 00:00:00

总数量 3 张

剩余 0 张

所有商店均可用

类型 蓝鲸券

发布时间 2024-06-07 16:48:33

过期时间 2024-06-30 00:00:00

总数量 5 张

剩余 5 张

查看优惠券组

蓝鲸在线购物

顾客版

用户头像

菜单

搜索

购物车

电源

查看我领取的优惠券

可用商店：手办商铺 领取

类型 满减券

优惠 满 100 减 20

发布时间 2024-06-06 20:23:45

过期时间 2024-06-17 00:00:00

总数量 10 张

剩余 7 张

可用商店：手办商铺 领取

类型 蓝鲸券

发布时间 2024-06-06 20:24:06

过期时间 2024-06-18 00:00:00

总数量 10 张

剩余 7 张

所有商店均可用 已领光! 下次早点来~

类型 代金券

发布时间 2024-06-06 20:24:30

过期时间 2024-06-13 00:00:00

总数量 2 张

剩余 0 张

可用商店：手办商铺 已过期!

类型 代金券

发布时间 2024-06-06 20:37:38

过期时间 2024-06-05 00:00:00

总数量 3 张

剩余 3 张

可用商店：手办商铺 已领光! 下次早点来~

类型 普通打折券

发布时间 2024-06-06 22:09:49

过期时间 2024-06-18 00:00:00

总数量 3 张

剩余 0 张

所有商店均可用 领取

类型 蓝鲸券

发布时间 2024-06-07 16:48:33

过期时间 2024-06-30 00:00:00

总数量 5 张

剩余 5 张

可用商店：终末番 领取

类型 代金券

发布时间 2024-06-07 16:56:57

过期时间 2024-06-18 00:00:00

总数量 10 张

剩余 10 张

可用商店：终末番 领取

类型 普通打折券

发布时间 2024-06-07 16:57:25

过期时间 2024-06-16 00:00:00

总数量 5 张

剩余 5 张

使用优惠券支付

查看我领取的优惠券

领取成功!

可用商店: 手办商铺

领取

类型: 满减券

优惠: 满 100 减 20

发布时间: 2024-06-06 20:23:45

过期时间: 2024-06-17 00:00:00

总数量: 10 张

剩余: 7 张

可用商店: 手办商铺

领取

类型: 蓝鲸券

发布时间: 2024-06-06 20:24:06

过期时间: 2024-06-18 00:00:00

总数量: 10 张

剩余: 7 张

所有商店均可用

已领光! 下次早点来~

类型: 代金券

发布时间: 2024-06-06 20:24:30

过期时间: 2024-06-13 00:00:00

总数量: 2 张

剩余: 0 张

可用商店: 手办商铺

已过期!

类型: 代金券

发布时间: 2024-06-06 20:37:38

过期时间: 2024-06-05 00:00:00

总数量: 3 张

剩余: 3 张

可用商店: 手办商铺

已领光! 下次早点来~

类型: 普通打折券

发布时间: 2024-06-06 22:09:49

过期时间: 2024-06-18 00:00:00

总数量: 3 张

剩余: 0 张

所有商店均可用

已领取!

类型: 蓝鲸券

发布时间: 2024-06-07 16:48:33

过期时间: 2024-06-30 00:00:00

总数量: 5 张

剩余: 4 张

可用商店: 终末番

已领取!

类型: 代金券

发布时间: 2024-06-07 16:56:57

过期时间: 2024-06-18 00:00:00

总数量: 10 张

剩余: 9 张

可用商店: 终末番

已领取!

类型: 普通打折券

发布时间: 2024-06-07 16:57:25

过期时间: 2024-06-16 00:00:00

总数量: 5 张

剩余: 4 张

蓝鲸在线购物

顾客版

用户图标

列表图标

搜索图标

购物车图标

电源图标

新建订单

神子v2.0

品类: 食品

价格: 20 元

库存: 981 件

提货方式: 请选择

商品数量: 1

订单总价: 20 元

创建订单

订单支付

不使用优惠券

购买数量	1		
提货方式	到店自提		
总价	19		
☒	序号	优惠券种类	截止日期
☒	1	蓝鲸券	2024-06-30 00:00:00

确认支付

稍后支付

订单号 12

已完成

商品 早柚

数量 5 件

总价 60 元

取货类型 到店自提

创建时间 2024-06-07 17:52:03

查看

订单号 13

待商家发货

商品 神子v2.0

数量 1 件

总价 20 元

取货类型 到店自提

创建时间 2024-06-07 18:03:37

查看

其他功能

测试内容	预期结果	测试结果
查询商品	商品查询成功	正确
下载订单报表	订单报表在本地下载成功	正确

查询商品

商品名称

商店名称

品类

请选择

最小价格

最大价格

查询

重置

商品列表



神子v1.0

品类 奢侈品

价格 15 元

库存 982 件

参与评分 3 人

★ ★ ★ ★ ★



神子v2.0

品类 食品

价格 20 元

库存 983 件

参与评分 0 人

★ ★ ★ ★ ★



早柚

品类 娱乐

价格 12 元

库存 986 件

参与评分 0 人

★ ★ ★ ★ ★

商品名称

商店名称

品类

食品

最小价格

最大价格

查询

重置

商品列表



神子v2.0

品类 食品

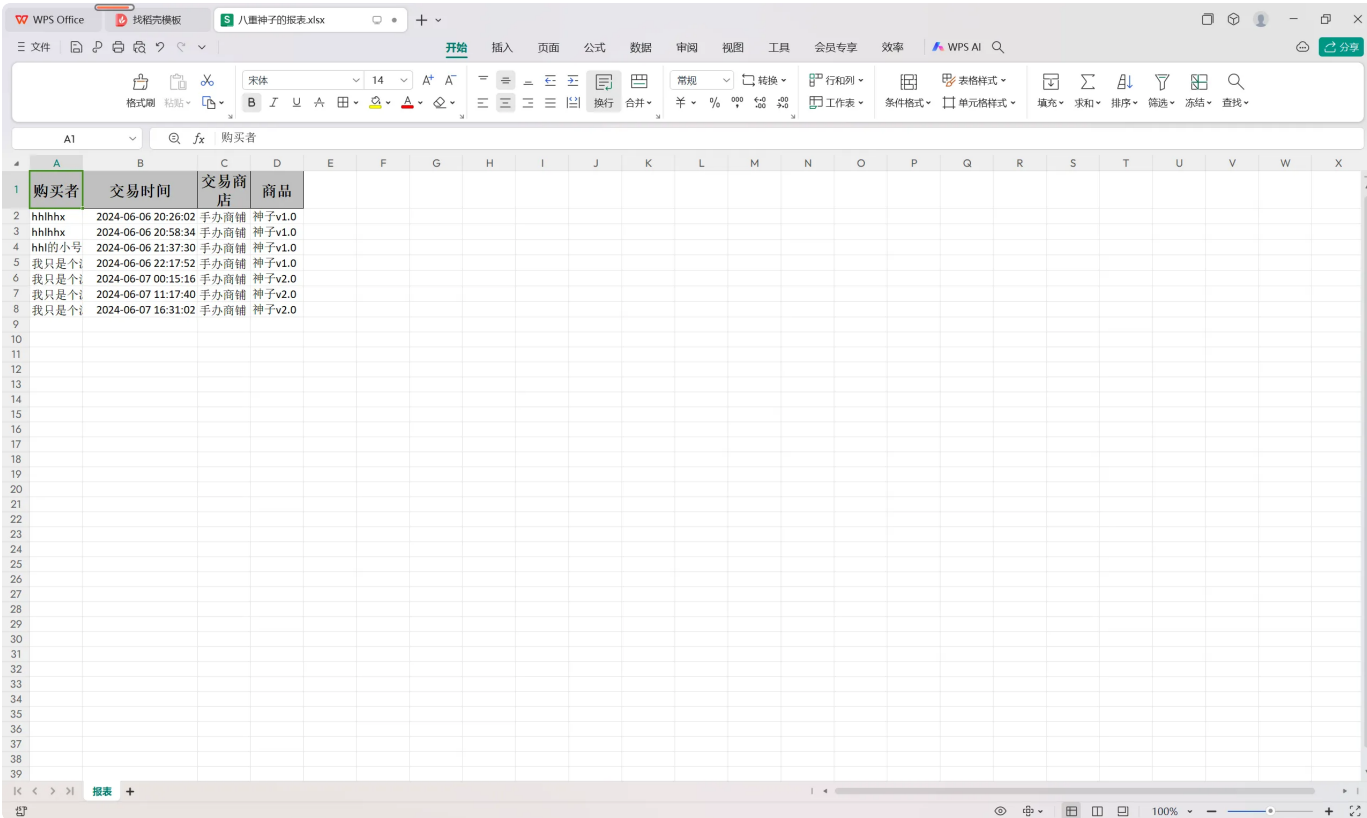
价格 20 元

库存 983 件

参与评分 0 人

★ ★ ★ ★ ★

下载订单报表



自主拓展

测试内容	预期结果	测试结果
------	------	------

评论回复	评论回复成功，且正常显示	正确
多种优惠券	多种优惠券可正常发布与使用	正确
优惠券可叠加	支付时可叠加使用优惠券	正确

评论回复

蓝鲸在线购物

顾客版

用户图标

购物车图标

搜索图标

收藏夹图标

电源图标

←



神子v1.0

品类 奢侈品

价格 15 元

库存 982 件

购买

评论区 (点击评论展开详情)

hhlhxx

2024-06-06 21:21:56

★★★★★

这个商品真好看www，必须打满分!!!

hhl的小号

2024-06-06 21:23:30

回复 hhlhxx : 真的嘛!!! 那我也准备买了!!!

hhlhxx

2024-06-06 22:12:37

回复 hhl的小号 : 真的!!!

我只是个游客

2024-06-06 22:17:02

回复 hhlhxx : 那我也买!

回复 hhl的小号 :

发表评论

hhl的小号

2024-06-06 21:39:02

★★★★★

真的很好看诶!!!

蓝鲸在线购物

顾客版

用户图标

购物车图标

搜索图标

收藏夹图标

电源图标

←



神子v1.0

品类 奢侈品

价格 15 元

库存 982 件

购买

评论区 (点击评论展开详情)

hhlhxx

2024-06-06 21:21:56

★★★★★

这个商品真好看www，必须打满分!!!

hhl的小号

2024-06-06 21:23:30

回复 hhlhxx : 真的嘛!!! 那我也准备买了!!!

hhlhxx

2024-06-06 22:12:37

回复 hhl的小号 : 真的!!!

我只是个游客

2024-06-06 22:17:02

回复 hhlhxx : 那我也买!

我只是个游客

2024-06-07 16:55:12

回复 hhl的小号 : 确实不错诶!!!

hhl的小号

2024-06-06 21:39:02

★★★★★

真的很好看诶!!!

多种优惠券

蓝鲸在线购物商家版

新建优惠券组

优惠券类型

请选择

优惠券数量

满减券

蓝鲸券

代金券

普通打折券

过期时间

创建优惠券组

蓝鲸在线购物商家版

新建优惠券组

优惠券类型

代金券

优惠

5

元

优惠券数量

10

过期时间

2024-06-18

创建优惠券组



新建优惠券组

优惠券类型 普通打折券

折扣为原价的：80%

优惠券数量 5

过期时间 2024-06-16

创建优惠券组

创建优惠券组

可用商店：终未番	可用商店：终未番
类型 代金券	类型 普通打折券
发布时间 2024-06-07 16:56:57	发布时间 2024-06-07 16:57:25
过期时间 2024-06-18 00:00:00	过期时间 2024-06-16 00:00:00
总数量 10 张	总数量 5 张
剩余 10 张	剩余 5 张

优惠券可叠加



订单号 12	已完成	订单号 13	待商家发货	订单号 15	待商家发货
商品	早柚	商品	神子v2.0	商品	早柚
数量	5 件	数量	1 件	数量	3 件
总价	60 元	总价	20 元	总价	36 元
取货类型	到店自提	取货类型	到店自提	取货类型	到店自提
创建时间	2024-06-07 17:52:03	创建时间	2024-06-07 18:03:37	创建时间	2024-06-07 18:06:34
查看		查看		查看	

测试总结

1. 单元测试总结

测试覆盖率：测试用例覆盖了不同的方法和场景，包括创建店铺、获取店铺、获取所有店铺、获取评分和搜索产品等等。通过编写多个测试用例，可以确保代码在各种情况下的正确性和稳定性。

测试边界条件：在编写测试用例时，我尽量考虑了各种可能的边界条件，例如店铺已存在、店铺不存在、评论为空等情况。这有助于确保代码在边界情况下能够正确处理，并提高代码的鲁棒性。

模拟依赖对象：使用@MockBean注解来模拟依赖的存储库对象，使得测试可以独立于实际的存储库实现。这样可以更好地隔离和控制测试环境，提高测试的可靠性和可复现性。

验证方法调用：使用verify方法来验证存储库对象的方法是否按预期进行了调用，例如通过verify(storeRepository, times(1)).findByName("Test Store")来验证storeRepository.findByName方法是否被调用了一次。

断言结果：使用断言来验证方法的返回结果是否符合预期，例如通过Assertions.assertEquals(1, result.getId())来验证获取店铺的方法返回的ID是否正确。

通过编写这些单元测试，可以有效地验证代码的正确性和稳定性。单元测试是开发过程中的重要环节，它可以帮助发现潜在的问题并提高代码的质量。同时，单元测试还能够提供示例，方便其他开发人员理解和使用代码。除此之外，单元测试的好处是我们不必完成整个框架再进行测试，实现**quick failure**可以帮助我们提前发现一些问题，**避免浪费时间**在不必要的定位问题以及Debug。

2. 集成测试总结

集成测试的目标是验证多个组件或模块在一起工作时的协同能力。在系统开发的集成阶段，集成测试确保了各个模块在数据和功能上无缝对接，找出接口间可能存在的问题，从而保障系统整体的稳定性和一致性。通过模拟真实的业务场景，测试发现并修复了模块间的接口问题，保障了系统在多模块协同工作下的稳定性和一致性。未来，在系统扩展或新增模块时，仍需保持对集成测试的重视，以确保系统整体质量的持续提升。

3. 功能测试总结

在前端进行功能测试时会发现许多在后端开发时很难注意到的问题，如我们在进行功能测试的时候，发现了优惠券叠加使用时的计算方式有误，以及在评论回复时有一些细节问题，这些问题都是很难在编程过程中能意识到的，通过功能测试，我们及时修复了许多bug，提高了我们项目的可用性，同时反映了功能测试在项目开发过程中的重要地位。

附录